HEALTHCARE ASSISTANT

Arquitectura y demostración

Asistente virtual para una clínica

Python • FastAPI • LangGraph • RAG • MCP

FAQ

TURNOS

SAFETY

01 ¿Qué hace mi aplicación?

Una API backend que procesa consultas de pacientes.

Preguntas frecuentes

Responde consultas sobre horarios, servicios e información de la clínica.

Turnos médicos

Consulta disponibilidad y permite reservar horarios disponibles.

Seguridad

Identifica consultas sensibles y evita ofrecer diagnósticos.

No tiene frontend: se demuestra utilizando Swagger.

02 Arquitectura general

El recorrido de una consulta, de principio a fin.



03 FastAPI, Pydantic y LangGraph

Archivos: app/main.py · app/models/patient.py · app/graph/workflow.py

FastAPI

POST /chat recibe el mensaje y ejecuta healthcare_graph.ainvoke().

Pydantic

PatientRequest valida patient_id y message. Si faltan datos, responde 422.

LangGraph

StateGraph organiza los nodos y dirige la consulta a FAQ, Booking o Safety.

Para mostrar: abrí los archivos y señalá las funciones principales.

04 RAG: preguntas frecuentes

Archivos: data/clinic_faq.json · app/rag/embeddings.py · retriever.py · generator.py

1. FAQs

Se leen las preguntas y respuestas desde JSON.

2. Embeddings

OpenAI convierte el texto en vectores numéricos.

3. Similitud

Se compara la pregunta con las FAQs usando similitud coseno.

4. Respuesta

Score ≥ 0.65: GPT-4.1-mini responde con el contexto.

05 Caché y rendimiento

Archivo: app/rag/retriever.py

@lru_cache(maxsize=1)

Primera consulta FAQ

Se calculan los embeddings de las FAQs y se guardan en memoria.

Consultas siguientes

Se reutilizan las FAQs en caché; cada pregunta nueva genera su propio embedding.

El caché se pierde al reiniciar la aplicación.

06 MCP y reservas de turnos

Archivos: mcp_server/server.py · app/services/booking.py · data/doctor_schedule.json

check_availability

Consulta si un horario está disponible. Solo lee la información del JSON.

book_slot

Si el horario está libre, confirma la reserva y elimina ese turno de los disponibles.

FastMCP registra ambas herramientas mediante @mcp.tool().

07 Safety y pruebas automatizadas

Archivos: app/graph/nodes.py · tests/test_chat.py

Safety

Identifica emergencias y consultas médicas sensibles. Evita diagnósticos y tratamientos.

Pytest

Seis pruebas comprueban los endpoints, la validación 422 y las rutas FAQ, Booking y Safety.

python -m pytest tests/test_chat.py -v

08 Demostración en Swagger

Ejecutar: python -m uvicorn app.main:app --reload

Abrir la API

http://127.0.0.1:8000/docs

POST /chat → Try it out → Execute

Tres ejemplos

FAQ: What are the clinic opening hours?

Safety: I can’t breathe

Booking: consultar un turno real del JSON

Mostrar en la respuesta: patient_id, route y response.

09 Conclusión y próximos pasos

FastAPI recibe. Pydantic valida.
LangGraph decide.

RAG responde. MCP gestiona turnos.
Safety protege. Pytest verifica.

Próxima mejora: frontend con calendario de reservas.